

Arabic NLP Pipeline (POC) with Model Comparison Framework

Technical Documentation (Implementation & Repository Structure)

Final Year Engineering Project (PFE)

February 13, 2026

Contents

1	Project Overview	3
1.1	Core Objectives	3
1.2	Technology Stack	3
2	Repository Structure	3
2.1	Directory Layout	4
2.2	Run Output Folder	4
3	Pipeline Execution Flow	5
3.1	Data Flow	5
3.2	Clean Persistence Strategy	5
4	Database Tables (MySQL)	5
4.1	Output Format (Key Tables)	6
4.1.1	NER Results Table	6
4.1.2	Sentiment Results Table	6
4.1.3	Keyword Results Table	6
5	Arabic Preprocessing (src/preprocessing.py)	6
6	Language Detection (fastText)	7
7	Named Entity Recognition (NER)	7
7.1	Models Compared	7
7.2	Robust Chunking for Long Articles	7
7.3	Unified Label Scheme	7
8	Sentiment Analysis	7
8.1	Models Compared	7
8.2	Probability Normalization	8
9	Keyword Extraction (TF-IDF)	8
10	Timing Measurement	8

11 Model Comparison Framework (comparison/)	8
11.1 Motivation: No Labeled Data	8
11.2 Comparison Modules	9
11.3 Expected Outputs (CSV)	9
12 Reproducibility and Traceability	9
13 Limitations and Next Steps	9
13.1 Limitations	9
13.2 Next Steps	10

1 Project Overview

This project implements a Proof-of-Concept (POC) NLP pipeline for Arabic media content (Libyan news and social media text). The pipeline is designed to process large volumes of articles in a modular, reproducible manner.

1.1 Core Objectives

- Detect article language and filter only Arabic texts for downstream tasks.
- Run multiple Transformer models for:
 - 1) Named Entity Recognition (NER)
 - 2) Sentiment Analysis
- Extract keywords using TF-IDF (statistical method).
- Store all results persistently in MySQL (drop & recreate tables per run).
- Generate a separate, reusable model comparison report without modifying the pipeline logic.

1.2 Technology Stack

Component		Technology
Programming Language	Lan-	Python 3.11
Database		MySQL, SQLAlchemy
Language Detection		fastText (1id.176.ftz)
NER & Sentiment		Hugging Face Transformers + PyTorch
Keyword Extraction		scikit-learn TF-IDF
Logging & Reproducibility		Timestamped run folders with <code>run.log</code> and <code>run_config.json</code>

2 Repository Structure

The project is organized around two main layers:

- **Pipeline layer** (`src/` and `main.py`): fetch data, preprocess, infer, store outputs in MySQL.
- **Comparison layer** (`comparison/`): reads pipeline outputs and produces local CSV reports.

2.1 Directory Layout

Listing 1: High-level repository layout

```
NLP_MENA/  
|-- main.py  
|-- src/  
|   |-- config.py  
|   |-- db_config.py  
|   |-- preprocessing.py  
|   |-- language_detection.py  
|   |-- ner_extraction.py  
|   |-- sentiment_analysis.py  
|   |-- keyword_extraction.py  
|  
|-- comparison/  
|   |-- __init__.py  
|   |-- ner_comparison.py  
|   |-- sentiment_comparison.py  
|   |-- keyword_comparison.py  
|   |-- timing_comparison.py  
|   |-- report_generator.py  
|  
|-- models/  
|   |-- lid.176.ftz  
|  
|-- results/  
|   |-- YYYY-MM-DD_HH-MM-SS/  
|       |-- run.log  
|       |-- run_config.json  
|       |-- comparison.log  
|       |-- ner_comparison.csv  
|       |-- sentiment_comparison.csv  
|       |-- keyword_comparison.csv  
|       |-- timing_comparison.csv  
|  
|-- .env  
|-- requirements.txt
```

2.2 Run Output Folder

Each execution creates a unique timestamped folder under **results/**. This folder stores:

- **run.log** (pipeline execution log)
- **run_config.json** (full configuration + environment versions)
- **comparison.log** (comparison execution log)
- Four comparison CSV outputs

3 Pipeline Execution Flow

3.1 Data Flow

The pipeline executes the following stages:

- Step 1: Sampling from MySQL:** select a sample of articles from a raw table.
- Step 2: Arabic preprocessing:** generate task-specific text variants.
- Step 3: Language detection:** fastText predicts language + confidence; store to MySQL.
- Step 4: Arabic filtering:** only (`lang = "ar"`) with confidence above threshold continue.
- Step 5: NER inference:** run AraBERT vs CAMEL vs mBERT; store consolidated row per article.
- Step 6: Sentiment inference:** run AraBERT vs CAMEL vs mBERT; store consolidated row per article.
- Step 7: Keyword extraction:** TF-IDF fit on corpus and extract top-N per article; store to MySQL.
- Step 8: Local comparison report:** compute metrics and export CSVs to `results/<run>/`.

3.2 Clean Persistence Strategy

- The raw MySQL dataset is treated as **read-only**.
- Results are written into dedicated tables.
- At the start of each run, result tables are **dropped and recreated** to avoid mixing outputs across runs.

4 Database Tables (MySQL)

The pipeline writes seven result tables (recreated at each run):

Table	Description
<code>lang_detection</code>	One row per article: predicted language, confidence, expected language, correctness flag
<code>preprocess_ner</code>	One row per article: <code>text_ner</code> (preprocessed text for NER)
<code>preprocess_sentiment</code>	One row per article: <code>text_sentiment</code>
<code>preprocess_keywords</code>	One row per article: <code>text_keywords</code>
<code>ner_results</code>	One row per article: three columns for entity strings (AraBERT/CAMEL/mBERT)
<code>sentiment_results</code>	One row per article: labels, scores, and probability JSON for three models

Table	Description
<code>keyword_results</code>	One row per article: ranked keyword list with TF-IDF scores

Table 2: MySQL result tables recreated at each pipeline run.

4.1 Output Format (Key Tables)

4.1.1 NER Results Table

The NER output is stored as a **single row per article** with three model columns. Each column is a formatted list:

```
entity_text (LBL, score), entity_text (LBL, score), ...
```

Labels are unified to **three-letter codes** (first 3 letters uppercased), e.g. `ORGANIZATION` → `ORG`.

4.1.2 Sentiment Results Table

Sentiment stores one row per article with model-specific columns:

- `<model>_label`: POS/NEG/NEU (AraBERT may output MIX)
- `<model>_score`: best probability (rounded to 4 decimals)
- `<model>_probs`: JSON distribution (rounded to 4 decimals)

4.1.3 Keyword Results Table

Keywords are stored as one row per article:

```
word1 (0.4134), word2 (0.2756), ...
```

5 Arabic Preprocessing (src/preprocessing.py)

Arabic preprocessing is implemented via `ArabicPreprocessor`, using task-specific presets:

- **Language detection preset**: light cleaning (URLs, emails, HTML removal, NFC normalization).
- **NER & Sentiment presets**: more intensive cleaning (diacritics removal, tatweel removal, special-char filtering) while preserving linguistic features useful for BERT models.
- **Keywords preset**: aggressive normalization (remove numbers, reduce repeated chars) to improve TF-IDF quality.

A heuristic fix is included to split concatenations of “” when merged with a preceding Arabic token (useful for noisy news text).

6 Language Detection (fastText)

Language detection uses fastText `lid.176.ftz`. For each article:

- Predict `lang` and confidence score.
- Compare to database `id_language` mapping to compute a consistency flag.
- Store results in `lang_detection`.
- Only Arabic articles above a confidence threshold pass to NLP inference.

7 Named Entity Recognition (NER)

7.1 Models Compared

- **Model 0 (AraBERT)**: `hatmimoha/arabic-ner`
- **Model 1 (CAMEL)**: `CAMEL-Lab/bert-base-arabic-camelbert-msa-ner`
- **Model 2 (mBERT)**: `Davlan/bert-base-multilingual-cased-ner-hrl`

7.2 Robust Chunking for Long Articles

Transformer models have a 512-token limit. The NER module applies:

- Token-based chunking using tokenizer offset mappings.
- Overlap between chunks to avoid losing boundary entities.
- Deduplication and merge logic to reduce overlap artifacts.

7.3 Unified Label Scheme

Different models output different tags (e.g. `ORGANIZATION` vs `ORG`). The implementation unifies labels by mapping to **three-letter uppercase codes**.

8 Sentiment Analysis

8.1 Models Compared

- AraBERT sentiment: 4-class (`POS/NEG/NEU/MIX`)
- CAMEL sentiment: 3-class (`POS/NEG/NEU`)
- mBERT (`nlptown`): 1–5 stars mapped into `POS/NEG/NEU`

8.2 Probability Normalization

Each model output distribution is normalized to a comparable schema:

- AraBERT: maps `Positive/Negative/Neutral/Mixed` to `POS/NEG/NEU/MIX`
- CAMEL: maps `positive/negative/neutral` to `POS/NEG/NEU`
- nlptown: maps stars to 3-class via:

$$NEG = P(1) + P(2), \quad NEU = P(3), \quad POS = P(4) + P(5)$$

9 Keyword Extraction (TF-IDF)

Keywords are extracted using scikit-learn `TfidfVectorizer`. The vectorizer is fitted on the Arabic corpus of the current run and returns the top-N keywords per article after stopwords removal (Arabic stopwords defined in `Config.ARABIC_STOPWORDS`).

10 Timing Measurement

Because this is a POC with no labeled data, model comparison includes technical performance. The pipeline measures inference time using `time.perf_counter()`:

- For each article and each model call: record start/end time and store elapsed seconds in an in-memory list.
- The comparison layer aggregates these records into `timing_comparison.csv`.

The timing CSV contains:

- `task`: `ner`, `sentiment`, `keywords`
- `model_name`: `arabert`, `camel`, `mbert`, `tfidf`
- `total/mean/std/min/max` time per article

11 Model Comparison Framework (comparison/)

11.1 Motivation: No Labeled Data

Since the dataset is real news content and has no ground truth annotations, classical supervised metrics (accuracy, precision, recall, F1) cannot be computed. The evaluation is therefore based on:

- **Indirect quantitative signals** from outputs (counts, distributions, confidence).
- **Model agreement** (consistency across models).
- **Technical performance** (inference time).

11.2 Comparison Modules

- `ner_comparison.py`: entity volume, per-article averages, label distribution (PER/LOC/ORG/EV), confidence statistics, agreement and uniqueness indicators.
- `sentiment_comparison.py`: label distributions, confidence statistics, agreement indicators, unanimous agreement percent.
- `keyword_comparison.py`: average keywords per article, unique keywords, top frequent keywords.
- `timing_comparison.py`: aggregates timing records into descriptive statistics.
- `report_generator.py`: orchestrates all comparisons, writes CSV outputs and `comparison.log` in the run folder.

11.3 Expected Outputs (CSV)

At the end of each run, the system produces:

- `ner_comparison.csv`
- `sentiment_comparison.csv`
- `keyword_comparison.csv`
- `timing_comparison.csv`

All of these are saved inside the run folder under `results/`.

12 Reproducibility and Traceability

The run folder includes:

- `run_config.json`: captures model IDs, parameters, thresholds, sample size, and package versions.
- `run.log` and `comparison.log`: provide step-by-step trace of the pipeline and comparison computations.

This design enables reproducible experiments and allows auditing the pipeline behavior without modifying raw data.

13 Limitations and Next Steps

13.1 Limitations

- No labeled data, thus no direct accuracy/F1 measurement.
- Agreement and confidence can be misleading (models can agree on incorrect outputs).
- Results tables are recreated per run (no historical tracking across runs).

13.2 Next Steps

- Add optional run persistence (store results with a run ID instead of dropping tables) for longitudinal comparisons.
- Add lightweight manual evaluation protocol (small annotated subset) to estimate correctness rates.
- Optimize throughput using batch inference and bulk inserts for very large datasets.